

Lettuce Eat

Incremental Backend Implementation Plan

Plan 1: Server-Owned Correction Loop

Objective: fastest path from photo to corrected macros to logged meal in under 10 seconds.

Product principle: Fast estimate -> Fast correction -> Logged meal.

MVP Scope

- Included: photo meal capture, AI food detection, AI macro estimation, quick correction support, and logged meal storage.
- Not included for MVP: barcode scanning, nutrition database integration, restaurant database, Apple Health, social features, and coaching.
- Success metric: Photo -> corrected macros -> logged meal in under 10 seconds.
- Primary goal: validate the interaction loop, not perfect nutrition accuracy.

Client Response Schema Requirement

- Canonical v1 analysis response sent to the client:
- analysis_id: string, for example analysis_123
- version: v1
- items[]: detected meal items with id, label, category, estimated_grams, confidence, macro_estimate, uncertainty, and assumptions.
- macro_estimate: calories, protein_g, carbs_g, fat_g.
- uncertainty: portion_size, ingredient_composition, cooking_fat.
- assumptions[]: key, label, value records used by the correction UI.
- meal_totals: calories, protein_g, carbs_g, fat_g aggregated across all items.
- overall_confidence: meal-level confidence score.
- suggestions[]: UI correction nudges with id, target_type, target_id, kind, direction, title, subtitle, and preview_delta.
- preview_delta: macro change preview for calories, protein_g, carbs_g, fat_g.
- This schema is the public API contract for the initial photo analysis response.
- Correction and log endpoints should return the same macro field names and item IDs so the client can update UI state without remapping.
- The validated JSON stored for the future dataset should preserve this client-facing schema plus internal metadata needed for audit/debugging.

Dataset And Storage Requirement

- The backend must preserve the full meal logging trail so it can become a training and evaluation dataset later.
- Store the original image or a durable object-storage key for the original image.
- Store the AI raw response exactly as returned by the provider for debugging, prompt iteration, and offline review.
- Store the validated JSON produced from the AI response after schema validation, using the v1 client response schema as the canonical app-level shape.
- Store every user correction applied during the correction loop, including values and timestamps.
- Store the final saved meal with final corrected macros, detected foods, suggestions accepted/ignored when available, and a link back to the original draft analysis.
- Do not use this data for training/evaluation without future privacy, consent, retention,

and deletion rules. MVP should preserve the data shape while keeping policy decisions explicit.

Real Image Testing Approach

- Add a small manual/integration test path that uses a real meal photo to validate the end-to-end backend loop.
- Keep normal automated tests mocked and deterministic so CI does not require OpenAI credentials or a checked-in private image.
- Use a local ignored folder such as test-assets/manual/ for real meal photos, or document a curl command that points to a user-provided image path.
- Real-image acceptance check: upload photo, receive a v1 response with items, meal_totals, overall_confidence, and suggestions; apply at least one correction; log the meal; confirm the whole flow can complete in under 10 seconds.
- Do not treat the real-image output as a golden nutrition truth; validate response shape, latency, correction behavior, successful logging, and dataset persistence fields.

Stage 0: Document The Target API Contract

- Purpose: lock the backend shape before changing code.
- Add docs/meal-flow-api.md describing analyze photo, apply corrections, and log meal.
- Define draft vs logged meal states.
- Document the v1 client response schema exactly: analysis_id, version, items, meal_totals, overall_confidence, and suggestions.
- Document allowed item category, uncertainty, suggestion kind, and direction values used by the correction UI.
- Define example JSON responses for photo analysis, corrected macros, and logged meals using the v1 schema field names.
- Document the dataset record created by each flow: original image, AI raw response, validated JSON, user corrections, and final saved meal.
- Document how to run a manual real-image test with a local meal photo and OpenAI API key.
- Deliverable: documentation only; no production code changes.

Stage 1: Add New Schemas Without New Behavior

- Purpose: introduce the data model safely before wiring endpoints.
- Add Pydantic schemas matching the v1 response: MealAnalysisResponse, MealItem, MacroEstimate, Uncertainty, Assumption, MealTotals, Suggestion, and PreviewDelta.
- Use analysis_id, version, items, meal_totals, overall_confidence, and suggestions as the exact top-level response fields.
- Add request/response schemas for corrections and logged meals that keep item IDs stable across analyze, correction, and log steps.
- Validated JSON schemas should represent the canonical app-level interpretation of the AI response.
- Keep existing /nutrition/analyze behavior unchanged.
- Tests: schema validation for required top-level fields, item IDs, grams, macros, confidence ranges, suggestions, preview deltas, correction payloads, and persisted dataset metadata.

Stage 2: Extend AI Output Shape

- Purpose: make AI analysis return the v1 food-level structured response.
- Update the AI JSON schema and prompt to return detected items, estimated grams, item macro_estimate, uncertainty, assumptions, meal_totals, overall_confidence, and suggestions.
- Ensure suggestions are correction-ready nudges with target_type, target_id, kind, direction, title, subtitle, and preview_delta.
- Capture both the provider raw response and the validated JSON derived from it.
- Keep the current endpoint stable while the richer service method is introduced behind tests.
- Update fake AI service with deterministic food-level v1 test data.
- Add an optional manual real-image smoke test command that exercises the real OpenAI call outside CI.
- Tests: mocked AI response validates into new schemas; bad or missing AI fields fail validation; manual real-image run validates shape, latency, and raw/validated response capture.

Stage 3: Add Draft Meal Storage

- Purpose: persist analysis drafts separately from final logged meals and preserve dataset inputs.
- Add a draft meal model/table with status, image storage key or image bytes reference, validated v1 response JSON, detected items JSON, original meal_totals, current corrected totals, suggestion/correction history, and AI raw response.
- Prefer a new model/table over stretching the existing NutritionAnalysis model too far.
- Keep existing endpoint working until the new meal endpoint is ready.
- Tests: create and retrieve draft records; verify original image reference, raw AI response, validated v1 JSON, and corrected totals are persisted.

Stage 4: Add POST /api/v1/meals/analyze-photo

- Purpose: introduce the new MVP endpoint.
- Endpoint accepts image upload plus optional notes.
- Runs AI once, stores the original image or image reference, raw AI response, validated v1 JSON, and a draft meal analysis.
- Returns the v1 client response schema: analysis_id, version, items, meal_totals, overall_confidence, and suggestions.
- Existing /nutrition/analyze can remain for backward compatibility during transition.
- Tests: valid mocked image returns v1 draft analysis; unsupported upload returns 415; empty image returns 400; oversized image returns 413; draft and dataset fields are persisted; manual real-image curl returns the same response shape.

Stage 5: Add Deterministic Correction Engine

- Purpose: make corrections instant and server-owned without another AI call.
- Add a correction service that recalculates macros from draft data using item IDs and suggestion preview deltas where applicable.
- Support initial MVP correction controls: portion scale, composition ratio such as rice vs meat, and oil level.
- Append every correction request and resulting macro total to correction history for future

evaluation.

- Tests: correction math unit tests; verify corrections do not call AI; verify corrected totals and correction history are persisted; manual real-image run confirms corrections update the AI-derived draft instantly.

Stage 6: Add PATCH /api/v1/meals/{analysis_id}/corrections

- Purpose: expose the correction loop to the UI.
- Endpoint accepts correction payload targeting analysis_id and item IDs from the v1 response.
- Loads the draft analysis, applies the deterministic correction engine, persists updated draft values, and returns updated items, meal_totals, overall_confidence, and suggestions as appropriate.
- Persist user corrections as first-class dataset events linked to the draft analysis.
- API docs and examples should show portion, ratio, oil-level, and suggestion-based correction payloads.
- Tests: valid correction updates macros; missing analysis returns 404; logged meal cannot be corrected unless explicitly allowed; invalid correction values return 422; correction history is stored.

Stage 7: Add POST /api/v1/meals/{analysis_id}/log

- Purpose: complete Photo -> corrected macros -> logged meal.
- Endpoint converts a corrected draft into a logged meal.
- Store final macros, final items, correction history snapshot, and references back to the original image, AI raw response, and validated v1 JSON.
- Mark draft as logged or create a separate logged meal record.
- Prevent duplicate logging by default.
- Tests: log draft succeeds; already logged draft returns 409; missing analysis returns 404; logged meal stores final corrected macros and complete dataset linkage; manual real-image flow confirms photo -> corrected macros -> logged meal under 10 seconds.

Stage 8: Retire Or Deprecate Old Nutrition Endpoint

- Purpose: simplify the API surface after the new meal flow is working.
- Decide whether /api/v1/nutrition/analyze remains as legacy, is conceptually redirected to the new flow, or is removed before production.
- If legacy remains, decide whether it also writes the dataset fields or is excluded from the training/evaluation dataset.
- Update README and tests accordingly.
- Tests: if kept, legacy endpoint still works; if removed, tests focus only on the new meal flow.

Recommended Commit Structure

- docs: define meal logging API flow, v1 client schema, and dataset record
- schemas: add meal analysis response, correction, logged meal, and dataset models
- ai: return v1 food-level meal estimates and capture raw responses
- db: persist draft meal analyses and dataset fields
- api: add photo analysis meal endpoint with v1 response
- corrections: add deterministic macro adjustment service

- api: add meal correction endpoint and correction history
- api: add logged meal endpoint with dataset linkage
- tests: document and add optional real-image smoke test workflow
- docs: document full MVP meal logging and dataset flow